
Thinking with Patterns: Breaking the Perceptual Bottleneck in Visual Planning via Pattern Induction

Yichang Jian

State Key Lab of CAD& CG
Zhejiang University
mtdickens1998@gmail.com

Boyuan Xiao

State Key Lab of CAD& CG
Zhejiang University
xiaoby202@gmail.com

Zhenyuan Huang

State Key Lab of CAD& CG
Zhejiang University
jerryhzy@outlook.com

Yifei Peng

State Key Lab of CAD& CG
Zhejiang University
pengyf@zju.edu.cn

Yao-Xiang Ding

State Key Lab of CAD& CG
Zhejiang University
dingyx.gm@gmail.com

Abstract

Planning from raw visual input remains a significant challenge for current Vision-Language Models (VLMs), when the complexity of input is beyond their one-step perception capability. Motivated by recent advances in Thinking with Images (TWI), a reasonable solution is to decompose the perception process into simpler steps by iteratively acquiring and incorporating local visual evidence. However, even though current VLMs are well-trained in general TWI ability, their perceptual bottleneck in the planning domain remains. To tackle this challenge, we formulate TWI as a tool to gradually build and reflect an accurate internal world model. We find that the resulting training-free planning strategy enables VLMs to solve tasks that are far beyond their initial capabilities, at the cost that too many TWI operations would significantly increase the computational overhead. To further improve efficiency, we propose *Pattern Inference*, a novel TWI strategy enabling VLMs to actively recognize known visual patterns in the new tasks and directly infer local world model structures. To obtain these patterns, we propose *Pattern Induction*, an online inductive learning strategy treating visual patterns as *composite* and *reusable* experts, which are autonomously discovered and optimized from experience. Experimental evaluations in FROZENLAKE, CRAFTER and CUBEBENCH domains show that our approaches achieve a desirable balance between accuracy and efficiency. Project Page: <https://future-item.github.io/PI-TWI>.

1 Introduction

Due to remarkable reasoning abilities, Vision-Language Models (VLMs) have shown promising potentials in visual planning tasks [Driess *et al.*, 2023; Mei *et al.*, 2024; Feng *et al.*, 2025; Zhao *et al.*, 2025]. However, we observe that current VLMs could still struggle in planning domains with raw visual inputs, especially those with complicated relationships among a large number of target objects to be discovered and understood. In our experiments, we show that current VLMs can fail in challenging planning tasks even when provided with ground-truth planners. This suggests a major bottleneck beyond planning ability: The failure essentially lies in the fact that the complexity of input is beyond the capability of VLMs to perceive at once.

To tackle this perceptual bottleneck, we turn to recent advances in Thinking with Images (TWI) [OpenAI, 2025; Su *et al.*, 2025b]. This emerging paradigm is characterized by models leveraging visual information as intermediate steps in their thought process by generating intermediate visual cues in

the original input, or even predicting purely with visual reasoning traces [Luo *et al.*, 2025; Wu *et al.*, 2025; Li *et al.*, 2025a; Fu *et al.*, 2025; Duan *et al.*, 2025; Xu *et al.*, 2026]. The benefit of TWI could include various aspects, while one is particularly related to our problem, such that TWI can be utilized to break the visual perception process of VLMs into simpler steps [Liu *et al.*, 2024]. The key idea lies in iteratively simplifying the visual scenes during reasoning via image modification operations, making the VLMs attend to key aspects related to the current reasoning step. Through this strategy, the one-step perceptual bottleneck is significantly alleviated without directly enhancing the native visual perception ability of VLMs. It is natural to think that the similar idea can also be utilized in planning. Unfortunately, we observe that even though current VLMs are well-trained in general TWI ability, their perceptual obstacle in the planning domain remains (See Table 1).

To further explore potential solutions, we propose a novel formulation of TWI specially designed for planning tasks. In this formulation, we assume that VLMs are already armed with ground-truth planners in each task domain, functioning only as a tool to gradually build and reflect an accurate internal world model for running the planners. Specifically, we propose a training-free planning strategy that functions as follows: At the beginning, VLM is asked to build a noisy world model from global visual input. During the planning process, VLMs utilize TWI operators to iteratively attend to local parts of the visual input and conducts reflection on the correctness of the local world model structures. We find that given unlimited budget of TWI operations, VLMs are able to solve tasks that are far beyond their initial capabilities. On the other hand, we also recognize a fundamental potential issue: Too many TWI operations would significantly increase the computational overhead, which is measured by token usage in this paper.

To further improve efficiency, we propose *Pattern Inference*, a novel TWI strategy centered on the concept of *visual patterns*. A visual pattern represents a specific regularity for a local part of the visual input that (1) appears frequently among all inputs from one domain; (2) has a critical impact on the planning results. Assuming that VLMs keep a well-constructed library of such *reusable* and *composite* patterns, during planning, they can utilize the matching operation to actively recognize known visual patterns in all tasks in the domain and directly infer local world model structures. This operation has much lower cost than the naive structure reflection TWI operation, hence can be utilized to effectively address the previous concern on efficiency.

To build such pattern libraries, we propose *Pattern Induction*, an inductive learning strategy inspired by recent advances in neuro-symbolic inductive learning [Shindo *et al.*, 2023, 2024; Piriyaikulij *et al.*, 2025]. Based on the mixture of experts (MoE) [Jacobs *et al.*, 1991] and self-supervised learning by random masking [He *et al.*, 2022] frameworks, the method establishes an online learning strategy that continuously proposes new patterns by VLMs, reweights them using masked past trajectories, and exploits them in pattern inference to improve the efficiency of TWI for planning, allowing visual patterns to be autonomously discovered, optimized, and utilized from experience.

To summarize, we frame our paper as a proof-of-concept work, targeting at the justification of the following arguments: (1) Formulating TWI as a tool to gradually build and reflect an accurate internal world model could enable VLMs to effectively break their perceptual limitations in planning; (2) Visual patterns could be utilized as reusable and composite tools to significantly reduce the cost of building the internal world model; (3) Critical visual patterns could be autonomously constructed by VLMs based on the inductive online learning framework. We pack our approaches as *Pattern-Induced Thinking with Images* (PI-TWI) and evaluate it under three challenging visual planning domains, FROZENLAKE, CRAFTER, and CUBEENCH. The results show that our approaches achieve a desirable balance between accuracy and efficiency, empirically justifying the above arguments.

2 Related Work

2.1 Planning with VLMs

VLM-based planning has evolved from grounding language goals in visual observations to increasingly structured forms of long-horizon reasoning. Early embodied multimodal models, such as PaLM-E [Driess *et al.*, 2023], show that visual observations and language instructions can be integrated to support high-level planning for sequential robotic manipulation. Later approaches introduce closed-loop visual feedback, enabling planners such as ReplanVLM [Mei *et al.*, 2024] to detect execution failures and revise plans accordingly. More recent methods further move toward world-model-style reasoning: Reflective Planning [Feng *et al.*, 2025] imagines future world states to refine

long-horizon manipulation plans, while CoT-VLA [Zhao *et al.*, 2025] predicts future visual states as intermediate goals before action generation. This line of work has progressively improved the generation, revision, and evaluation of plans. However, this line of work mostly focuses on improving the planning process itself, while assuming that the visual information needed for planning can be reliably extracted from the scene. In contrast, our method targets this perceptual bottleneck by constructing a planner-sufficient world model from local visual evidence and learning reusable visual patterns to impute or prioritize task-critical facts with fewer costly VLM perception calls.

2.2 Thinking with Images

Recent advancements have expanded chain-of-thought (CoT) reasoning [Wei *et al.*, 2022] into the multimodal domain, giving rise to the Thinking of Images (TWI) paradigm. In this broader context, models leverage visual intermediates for a range of cognitive functions. These include invoking visual cropping tools to dynamically acquire evidence [Wu and Xie, 2024; Wang *et al.*, 2025; Li *et al.*, 2025b; Zheng *et al.*, 2026], utilizing visual scratchpads [Surís *et al.*, 2023; Hu *et al.*, 2024; Su *et al.*, 2025a; Fu *et al.*, 2025], and simulating future states directly in the visual modality [Li *et al.*, 2025a; Xu *et al.*, 2026]. Our work builds on this broader view, but focuses on the planning semantics of TWI. In our setting, the use of visual intermediates is to build a symbolic world model that is sufficient for a downstream planner. This requirement introduces structure that is absent from general-purpose TWI: the agent must decide which visual intermediates are relevant to planning and when the current world model is sufficient. Therefore, for our purpose, we formulate TWI as planner-sufficient world model construction.

2.3 Inductive Learning for Knowledge Acquisition

Our pattern induction method shares similar spirit with recent efforts in inductive learning for acquiring reusable symbolic knowledge from data. Representative works include differentiable inductive logic programming approaches [Evans and Grefenstette, 2018; Shindo *et al.*, 2023, 2024]. To induce logical rules from data, these approaches adopt a neuro-symbolic hybrid pipeline, which generates candidate logical clauses from symbolic inference, meanwhile conducts differentiable optimization for learning their fitness weights to the data. Similar methodology is adopted in neural program synthesis methods [Ellis *et al.*, 2021; Jones *et al.*, 2023], where symbolic function libraries are gradually induced and optimized from data, and a neural prediction model is learned to utilize the functions in the library to address the downstream tasks. However, a major challenge lies in how the primitive symbolic knowledge, such as candidate logical clauses and symbolic functions, are generated. Recent efforts have utilized language models for generating the symbolic proposals [Jones *et al.*, 2025; Hsu *et al.*, 2025; Piriyaakulkij *et al.*, 2025]. Notably, Piriyaakulkij *et al.* [2025] considers to utilize programs as composite experts to represent the transition rules in world models. Although we tackle different tasks, the visual patterns in our approach can be treated as a kind of symbolic knowledge to strengthen the visual perception and understanding abilities of VLMs. We hope that our work could inspire further research in the area towards this direction.

3 Method

To break the perceptual limitation of VLMs in visual planning, we formulate Thinking with Images (TWI) as a tool of constructing a planner-sufficient symbolic world model from visual evidence (Sec. 3.1). For reducing the cost of world model construction, we introduce *Pattern Inference*, a novel TWI strategy enabling VLMs to actively recognize known visual patterns in planning tasks and directly infer local world model structures (Sec. 3.2). To obtain these composite and reusable patterns from learning, we propose *Pattern Induction*, an online inductive learning method for building the pattern library from experience (Sec. 3.3). We name this framework PI-TWI, short for *Pattern-Induced Thinking with Images*. Fig. 1 provides the overall illustration.

3.1 Thinking with Images as Planner-Sufficient World Model Construction

We consider a planning instance

$$x = (I, \mathcal{U}, \{y_u\}_{u \in \mathcal{U}}, \Pi, C, G) \quad (1)$$

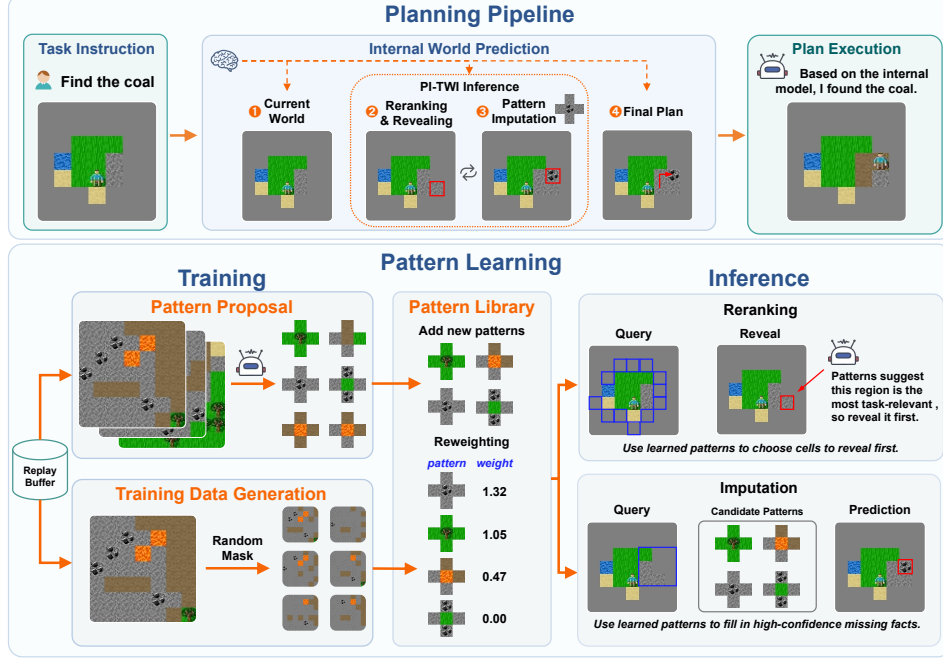


Figure 1: Overview of our PI-TWI approach.

where I is the input image, $\mathcal{U}$ is a finite set of visual variables, and each $u \in \mathcal{U}$ has a hidden value y_u in a finite domain $\mathcal{Y}_u$. A visual variable is the smallest unit that can be individually inspected by the TWI agent. For example, in CRAFT, a visual variable corresponds to a grid cell and its value is a tile or entity type, such as “grass”, “water”, or “stone”. In CUBEBENCH, a visual variable corresponds to a facelet on the cube and its value is one of the six colors, such as “yellow” or “red”. The true values $\{y_u\}_{u \in \mathcal{U}}$ are not available to the agent and must be grounded from the image or inferred from patterns.

A perception operation grounds one selected visual variable through a VLM call. We write

$$R(I, u) \rightarrow \hat{y}_u^R \quad (2)$$

for the reveal operator, which crops the image region containing u and grounds the observed content to a symbolic value. Each revealed value is a *directly grounded symbolic fact*. A *symbolic fact* is the basic unit of the symbolic world model. At step t , the agent maintains two sets of symbolic facts:

$$K_t^R = \{(u, \hat{y}_u^R) : u \text{ has been directly grounded by step } t\}, \quad (3)$$

$$K_t^I = \{(u, \hat{y}_u^I) : u \text{ has been imputed by pattern inference by step } t\}. \quad (4)$$

where K_t^R is the set of *directly grounded symbolic facts* which we have discussed above, and K_t^I is the set of *pattern-imputed symbolic facts* formulated in Eq. 10.

For a set of facts K , let $\text{dom}(K) = \{u : (u, \cdot) \in K\}$. The current symbolic world model is

$$M_t = K_t^R \oplus K_t^I = K_t^R \cup \{(u, \hat{y}_u^I) \in K_t^I : u \notin \text{dom}(K_t^R)\}, \quad (5)$$

where $\oplus$ basically means that **directly grounded facts override pattern-imputed facts whenever both are available for the same variable**, since per-variable perception is accurate, yet imputation is not.

The downstream planner for this task is denoted by Π . Since the planner may fail or return uncertified plans when the world model is incomplete, we assume an accompanying planner-sufficiency checker

$$C(M_t) \in \{0, 1\}. \quad (6)$$

The checker returns 1 when M_t contains enough information for Π to produce a plan satisfying the task criterion. In feasibility-oriented domains such as CRAFT and CUBEBENCH, sufficiency means that the current model supports a feasible plan. In optimality-oriented domains such as shortest-path planning in FROZENLAKE, sufficiency additionally requires that the returned plan can be certified as optimal regardless of the remaining unknown facts.

Our goal is to construct a sufficient and (largely) accurate world model with as few costly perception operations as possible. For each domain, we use a *policy-generation procedure* G as a grounding controller. Given the current world model M_t , G generates a set of candidate visual variables that are most worthy of being inspected next:

$$G(M_t) \subseteq \mathcal{U} \setminus \text{dom}(M_t) \quad \text{dom}(M_t) = \{u : (u, \cdot) \in M_t\} \quad (7)$$

where $\text{dom}(M_t)$ is basically the set of all visual variables in M_t .

Intuitively, G does not itself solve the planning task as a learned policy. Instead, it generates the next candidates that the agent might inspect in order to make the world model sufficient for planning. Given a rendered raw visual planning instance, an agent may actively inspect parts of the image, ground the inspected content into symbolic facts, and pass the resulting world model to a downstream planner. Since we focus on the perceptual part, we only consider the construction of the world model, and simply assume that this downstream planner is a ground-truth planner (Details are introduced in Appendix C.5), in the sense that given a correct world model, the planner would produce a correct solution. In our experiments, G is instantiated by LazySP [Dellin and Srinivasa, 2016] for FROZENLAKE, by a task-and-motion-planning-style search procedure [Garrett *et al.*, 2021] for CRAFT, and by random selection for CUBEBENCH. These procedures are deliberately task-specific: they reveal only variables that are relevant to the planner and associated checker, rather than asking the VLM to transcribe all variables at once.

Using G without pattern inference gives the ablation of our method without pattern inference, which we denote as PI-TWI *w/o inference*. This ablation repeatedly samples a variable $u_t \in G(M_t)$, reveals it through $R(I, u_t)$, updates M_t , and stops once $C(M_t) = 1$. It exploits the structure of each planning task, but it treats every instance independently and therefore cannot reuse patterns observed and induced from previous instances. This should be distinguished from *native TWI method*, such as OpenAI-style TWI [OpenAI, 2025], which choose their own tool-use and visual-inspection strategy in a largely task-agnostic manner. PI-TWI builds on the task-specific policy-generation procedure by learning a pattern library online and using it to impute or prioritize visual variables before spending additional costly VLM perception calls.

3.2 Pattern Inference as Gated Mixture-of-Experts

A symbolic pattern ρ is a *composite, reusable, conditional* regularity that can be used to predict the value of a target visual variable. We represent the pattern library as $\mathcal{L} = \{\rho_i\}_{i=1}^m$. Each pattern ρ_i consists of three components: (i) an applicability predicate $a_i(u) \in \{0, 1\}$ indicating whether the pattern can be used for target variable u ; (ii) a *gate* $g_i(u, M_t) \in \{0, 1\}$ indicating whether the required contextual facts of the pattern are present and matched in M_t ; and (iii) a prediction $\mu_i(u) \in \mathcal{Y}_u$ for the value of u when the gate is satisfied. This formulation covers both spatially local patterns, such as repeated tiles in a grid, and more structured patterns, such as facet-color constraints of a cube.

We view each pattern as a gated expert. For the target variable, if a pattern is applicable and its gate is satisfied, it contributes a smoothed categorical prediction:

$$q_i(v \mid u, M_t) = \begin{cases} 1 - \varepsilon, & v = \mu_i(u), \\ \frac{\varepsilon}{|\mathcal{Y}_u| - 1}, & v \neq \mu_i(u) \end{cases} \quad (8)$$

where ε is a small smoothing constant, ensuring non-zero probability. Let w_i be the non-negative reliability weight of pattern i . The set of active experts for target variable u is

$$\mathcal{A}(u, M_t) = \{i : a_i(u)g_i(u, M_t) = 1\}. \quad (9)$$

If at least one pattern is active, the pattern library predicts the weighted average of all active ones:

$$p_w(v | M_t) = \frac{\sum_{i \in \mathcal{A}(u, M_t)} w_i q_i(v | u, M_t)}{\sum_{i \in \mathcal{A}(u, M_t)} w_i} \quad (10)$$

If no pattern is active, the pattern library will fall back to the uniform distribution over $\mathcal{Y}_u$. For each v , $p_w(v | M_t) = \frac{1}{|\mathcal{Y}_u|}$.

In conclusion, (i) only active experts influence the prediction, and (ii) the learned weights determine how much each active pattern should be trusted, relatively.

Pattern-based imputation. Given M_t , PI-TWI can add value prediction with high confidence to the world model M_t , without directly revealing them. For an unrevealed variable u , we define

$$\hat{y}_u^I = \arg \max_{v \in \mathcal{Y}_u} p_w(v | M_t), \quad c_u = \max_{v \in \mathcal{Y}_u} p_w(v | M_t). \quad (11)$$

where $\hat{y}_u^I$ is the predicted value of u , and c_u is the confidence. If $c_u \geq \tau$, where τ is a confidence threshold, we add $(u, \hat{y}_u^I)$ to the set of pattern-imputed symbolic facts K_t^I . Since imputed facts no longer need to be perceived, they can help reduce the number of VLM perception calls.

Pattern-based reranking. Patterns can also help decide which variable to reveal next. When the policy-generation procedure $G(M_t)$ returns multiple candidates, instead of revealing a randomly sampled one, PI-TWI assigns the candidates a task-dependent score derived from $p_w(y_u | M_t)$ and reveals the highest-scoring one. For example, in CRAFT, when the current task is to find iron, we rank candidates by $p_w(y_u = \text{iron} | M_t)$. Reranking doesn't impute unrevealed variables, but spends perception calls on variables that are more likely to help the planner.

3.3 Online Inductive Learning for Building Pattern Library

PI-TWI learns its pattern library online from previous planning instances. First, we start with an empty pattern library $\mathcal{L}$ and replay buffer $\mathcal{B}$. The replay buffer $\mathcal{B}$ contains final symbolic world models from past instances. During the process, the agent stores the final symbolic world model of each instance into the replay buffer, propose patterns, adjust their weights, and then use the pattern library for probability inference. As more instances are solved, the buffer provides increasingly informative evidence for pattern learning.

Pattern proposal by VLM We use a VLM as a flexible pattern proposer. Given a batch of symbolic world models sampled from $\mathcal{B}$, the VLM gets the batch as input, and propose feasible candidate *macro symbolic patterns*, which will be then parsed into multiple symbolic , deduplicated, and added to $\mathcal{L}$. Each valid macro symbolic pattern will be parsed into one or more symbolic patterns.

Training data generation by masking Inspired by previous work [He *et al.*, 2022], we construct training data for self-supervised learning by randomly masking symbolic facts from world models in $\mathcal{B}$. For each final world model $M^{(b)} \in \mathcal{B}$, when generating its r -th mask, we randomly mask a subset of it $H^{(b,r)}$ to obtain a partially observed model $\tilde{M}^{(b,r)}$, where $\tilde{M}^{(b,r)} = M^{(b)} \setminus H^{(b,r)}$. This procedure will finally result in a dataset $\mathcal{D} = \{(\tilde{M}^{(b,r)}, H^{(b,r)})\}$.

Pattern reweighting Once we have the proposed patterns added and training data $\mathcal{D}$ generated, we can infer the probability and perform maximum likelihood estimation to obtain the weights w :

$$w^* = \arg \max_w \sum_{(\tilde{M}^{(b,r)}, H^{(b,r)}) \in \mathcal{D}} \left[\sum_{(u, y_u) \in H^{(b,r)}} \log p_w(y_u | \tilde{M}^{(b,r)}) \right] \quad (12)$$

Table 1: Accuracy comparisons over baselines. G. Acc. and P. Acc. are short for grounding and planning accuracy respectively, both reported in percent. Three representative base models, GPT-5.4 [OpenAI, 2026], Gemini 3.1 Pro [Google DeepMind, 2026], and Qwen3 VL 235B A22B [et al., 2025] are introduced.

Environment	Model	VLM direct output		Native TWI		PI-TWI	
		G. Acc.	P. Acc.	G. Acc.	P. Acc.	G. Acc.	P. Acc.
FROZENLAKE	GPT-5.4	92.88	85.00	92.46	85.00	97.19	90.78
	Gemini 3.1 Pro	93.87	87.50	93.66	85.00	97.15	94.00
	Qwen3 VL 235B A22B	54.92	17.50	45.78	17.50	95.35	82.78
CRAFTER	GPT-5.4	46.39	0.00	44.28	0.00	100.00	100.00
	Gemini 3.1 Pro	26.71	0.00	48.78	41.67	100.00	100.00
	Qwen3 VL 235B A22B	0.00	0.00	22.08	0.00	100.00	100.00
CUBE BENCH	GPT-5.4	25.73	0.00	98.83	96.00	100.00	100.00
	Gemini 3.1 Pro	18.65	0.00	91.44	71.00	100.00	100.00
	Qwen3 VL 235B A22B	16.00	0.00	17.08	0.00	100.00	100.00

During learning, only the scalar pattern weights w are optimized, and the patterns proposed by the VLM are kept fixed until the next proposal round. Since the number of active patterns is relatively small in our tasks, we use L-BFGS for quicker optimization. The proposal, masking, and reweighting steps are repeated periodically during the online learning process, allowing PI-TWI to improve as the procedure goes on.

4 Experiment

We conduct experiments to justify the following points: (1) Does VLM perception bottleneck exist in challenging visual planning tasks, even with VLM’s native TWI abilities? (2) Does our formulation of TWI as a tool for world model construction effectively address the perceptual bottleneck? (3) Can our Pattern Inference method effectively reduce the cost of TWI operations? (4) Does our Pattern Induction method build effective and generalizable pattern library through online inductive learning?

Benchmarks. Our experiments focus on three visual planning domains with different structures and objectives: shortest-path planning in FROZENLAKE [Towers *et al.*, 2024], long-horizon resource acquisition and crafting in CRAFTER environment [Hafner, 2022], and Rubik’s Cube planning in CUBE BENCH [Gao *et al.*, 2025]. Details of the benchmarks are included in Appendix C.

Baselines and Ablations. We compare PI-TWI against the following baselines and ablations: (1) **VLM direct output** asks the VLM to directly transcribe the rendered image into a symbolic world model in a single pass. This baseline tests whether the model can recover the full symbolic state without active visual grounding. (2) **Native TWI** lets an off-the-shelf agentic TWI system decide how to inspect or reason over the image using its own tool-use loop, without our task-specific policy-generation procedure G , planner-sufficiency checker C , replay buffer $\mathcal{B}$, or pattern library $\mathcal{L}$. In our experiments, this corresponds to native systems such as GPT-5.4 with Code Interpreter and Qwen Agent. This baseline is largely task-agnostic: it actively uses visual tools, but it is not given the task-specific G used by PI-TWI. (3) **PI-TWI w/o inference** uses the same reveal operator R and task-specific policy-generation procedure G as PI-TWI, but disables pattern-based imputation and reranking. The planner can only use directly revealed facts. Since it also does not call a VLM for pattern proposal, its proposal-token cost is zero. (4) **PI-TWI w/o reweighting** uses the same pattern library as PI-TWI, but disables online optimization of the pattern weights. This tests whether dynamically reweighting patterns using masked trajectories from replay buffer is necessary.

Evaluation Metrics. To evaluate the performance of PI-TWI in planning tasks, we focus on both efficiency and accuracy: (1) **Efficiency.** We employ total token consumption as a proxy for efficiency, comprising *perception* and *proposal* tokens. Total and perception tokens are averaged over all episodes, while proposal tokens are averaged over all proposals. (2) **Accuracy.** We assess performance using two primary metrics: *planning accuracy* and *grounding accuracy*. Planning accuracy evaluates the ability to generate a sequence of actions that successfully finish the goal,

Table 2: Efficiency comparisons using GPT-5.4 [OpenAI, 2026]. Accuracy is reported in percent; token costs are reported as input/output token counts, with k denoting thousands.

Environment	Method	Grounding Acc $\uparrow$	Planning Acc $\uparrow$	Prop. token $\downarrow$	Perc. token $\downarrow$	Total token $\downarrow$
FrozenLake	PI-TWI	97.19	90.78	463.7/267.8	3.10k/158.1	3.19k/208.9
	w/o inference	100.00	100.00	–	5.46k/278.4	5.46k/278.4
	w/o reweight	100.00	100.00	471.1/268.2	5.46k/278.4	5.55k/329.4
Crafter	PI-TWI	100.00	100.00	2.73k/208.8	141.9k/3.86k	142.4k/3.90k
	w/o inference	100.00	100.00	–	224.5k/6.10k	224.5k/6.10k
	w/o reweight	100.00	100.00	2.73k/208.8	180.2k/4.90k	180.8k/4.94k
CubeBench	PI-TWI	100.00	100.00	1.56k/499.3	4.31k/244.9	4.45k/289.9
	w/o inference	100.00	100.00	–	4.75k/270.0	4.75k/270.0
	w/o reweight	100.00	100.00	1.56k/500.1	4.75k/270.0	4.89k/315.0

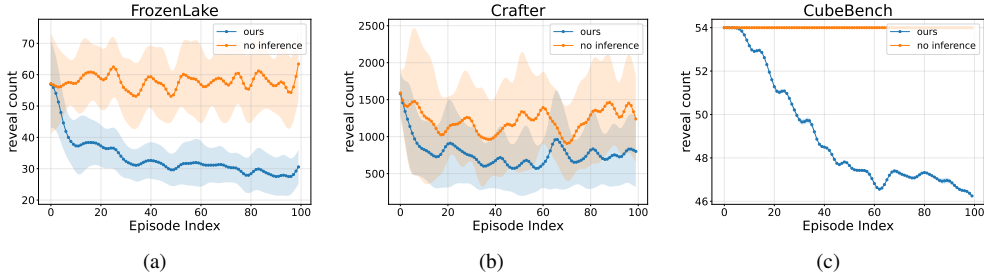


Figure 2: Reveal count comparisons in the efficiency experiment (gaussian smoothing, $\sigma = 2.0$).

measured as the ratio of successful episodes to total episodes. Grounding accuracy evaluates the ability to correctly represent the state space. It is defined as the ratio of correctly identified grids over the total number of grids:

$$\text{Grounding Accuracy} = \frac{N_{\text{correct_imp}} + N_{\text{correct_per}}}{N_{\text{imp}} + N_{\text{per}}} \quad (13)$$

where N_{imp} and N_{per} denote the total number of imputed and perceived visual variables, and $N_{\text{correct_imp}}$ and $N_{\text{correct_per}}$ represent the number of correctly imputed and perceived visual variables.

In summary, our evaluation covers five dimensions: perception/proposal/total tokens, and planning/-grounding accuracy. Note that we also use **reveal count** in our analysis, which only serves as a diagnostic measure to explain the source of token reduction, since under a fixed reveal operator, perception-token cost scales linearly with the number of perception calls; the main efficiency comparison is still based on total token consumption. We elaborate this matter further at Appendix B. Also, unless otherwise specified, quantitative comparisons in tables and experiment section are based on the sum of total input and output tokens.

4.1 Accuracy Comparison

As shown in Table 1, directly using VLM outputs leads to bad performance, illustrated by zero planning accuracy in visually complex or large-scale environments, like CUBEBENCH and CRAFTER. Native TWI improves performance for some environment-model pairs, but mainly on visually complex small-scale tasks like CUBEBENCH; however, its effectiveness is inconsistent across models and does not scale reliably to large-scale environments such as CRAFTER. This suggests that current TWI frameworks and the agentic behavior of foundation models may still be insufficient for these problems. In contrast, we formulate TWI-guided planning as a process of incrementally constructing a symbolic world model. Our full method, PI-TWI, is built upon this formulation. As shown in Table 1, PI-TWI consistently improves both grounding and planning accuracy across all evaluated tasks and models.

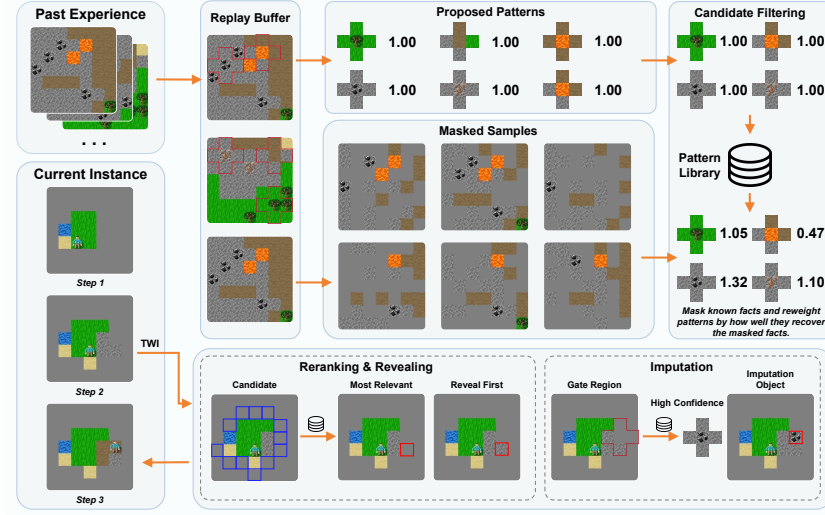


Figure 3: Qualitative illustration of the pattern inference and induction processes in CRAFTER.

4.2 Efficiency Comparison

As illustrated in Table 2, our method PI-TWI significantly outperforms alternative settings across all tasks in terms of total token consumption. In FROZENLAKE, although the planning accuracy with PI-TWI decreases to 90.78%, we argue that this represents a favorable trade-off between efficiency and accuracy. Specifically, we achieve a 40.77% reduction in total token consumption compared to the “w/o inference” baseline, at the expense of less than 10% decline in accuracy. Similarly, in CRAFTER, with the help of reranking, the total token consumption is also reduced to 63.44% of the “w/o inference” ablation. In CUBEBENCH, while the total token consumption only drops from 5.02k to 4.74k¹, this reduction is significant when considering the theoretical limit. Given that the theoretical minimum of reveal count is 46 (accounting for the 8 corner patterns) and associated minimum of 4.05k input perception tokens, our method achieves around 42.86% of the maximum possible reduction in terms of input perception tokens², which constitutes a substantial improvement in relative terms. The result shows that the reweighting mechanism is also crucial. Since VLM-proposed patterns are not always correct or equally useful, treating all patterns uniformly can cause noisy or irrelevant patterns to dominate the inference process. As shown in Table 2, removing reweighting substantially increases token usage, which justifies the effectiveness of our online inductive learning method.

Figure 2 further illustrates how these efficiency gains emerge during online learning. Overall, these trends support the need for online inductive learning: the pattern library becomes increasingly useful as more task trajectories are accumulated and put into learning.

4.3 Qualitative Results and Out-of-Distribution Generalization

Finally, we include a preliminary OOD generalizability study in Appendix A.1. The result suggests that patterns and associated weights learned on smaller maps can zero-shot transfer to larger maps. Furthermore, we provide qualitative illustrations of pattern inference and induction in Fig. 3 45.

5 Limitations and Future Work

As a proof-of-concept research, our work focuses on deterministic visual planning benchmarks rather than real-world or stochastic environments. We also assume that the set of possible variable types and the bounding box of each visual variable are known in advance. In this sense, our method addresses only one part of the broader TWI efficiency problem: how to reduce grounding cost when each

¹ $4.75k + 270 = 5.02k, 4.45k + 289.9 = 4.74k$

² $\frac{4.75 - 4.45}{4.75 - 4.05} = 42.86\%$. This is calculated using *input perception* tokens, instead of the sum of total input and output tokens.

reveal operation returns correct visual information and the symbolic interface is already specified. We plan to extend PI-TWI to real-world and stochastic environments by synergizing imputation and reranking. Specifically, *imputation* will continue to reduce grounding costs for high-certainty structural regularities (e.g., repeated architectural patterns). For noisier, non-deterministic regularities, *reranking* will serve as an interpretable heuristic to prioritize exploration and planning without committing to uncertain facts. This dual approach allows PI-TWI to exploit diverse environmental patterns while maintaining robustness to uncertainty.

Acknowledgement

This work was supported by National Natural Science Foundation of China (U23A20311,62206245).

References

- Christopher Dellin and Siddhartha Srinivasa. A unifying formalism for shortest path problems with expensive edge evaluations via lazy best-first search over paths with edge selectors. In *Proceedings of the international conference on automated planning and scheduling*, volume 26, pages 459–467, 2016.
- Danny Driess, Fei Xia, Mehdi S. M. Sajjadi, Corey Lynch, Aakanksha Chowdhery, Brian Ichter, Ayzaan Wahid, Jonathan Tompson, Quan Vuong, Tianhe Yu, Wenlong Huang, Yevgen Chebotar, Pierre Sermanet, Daniel Duckworth, Sergey Levine, Vincent Vanhoucke, Karol Hausman, Marc Toussaint, Klaus Greff, Andy Zeng, Igor Mordatch, and Pete Florence. Palm-e: An embodied multimodal language model, 2023.
- Chengqi Duan, Rongyao Fang, Yuqing Wang, Kun Wang, Linjiang Huang, Xingyu Zeng, Hongsheng Li, and Xihui Liu. Got-r1: Unleashing reasoning capability of mllm for visual generation with reinforcement learning. *arXiv preprint arXiv:2505.17022*, 2025.
- Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B Tenenbaum. Dreamcoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd acm sigplan international conference on programming language design and implementation*, pages 835–850, 2021.
- Shuai Bai et al. Qwen3-vl technical report, 2025.
- Richard Evans and Edward Grefenstette. Learning explanatory rules from noisy data. *Journal of Artificial Intelligence Research*, 61:1–64, 2018.
- Yunhai Feng, Jiaming Han, Zhuoran Yang, Xiangyu Yue, Sergey Levine, and Jianlan Luo. Reflective planning: Vision-language models for multi-stage long-horizon robotic manipulation, 2025.
- Xingyu Fu, Minqian Liu, Zhengyuan Yang, John Corring, Yijuan Lu, Jianwei Yang, Dan Roth, Dinei Florencio, and Cha Zhang. Refocus: Visual editing as a chain of thought for structured image understanding. *arXiv preprint arXiv:2501.05452*, 2025.
- Huan-ang Gao, Zikang Zhang, Tianwei Luo, Kaisen Yang, Xinzhe Juan, Jiahao Qiu, Tianxing Chen, Bingxiang He, Hao Zhao, Hao Zhou, et al. Cubebench: Diagnosing interactive, long-horizon spatial reasoning under partial observations. *arXiv preprint arXiv:2512.23328*, 2025.
- Caelan Reed Garrett, Rohan Chitnis, Rachel Holladay, Beomjoon Kim, Tom Silver, Leslie Pack Kaelbling, and Tomás Lozano-Pérez. Integrated task and motion planning. *Annual review of control, robotics, and autonomous systems*, 4(1):265–293, 2021.
- Google DeepMind. Gemini 3.1 pro model card. <https://deepmind.google/models/model-cards/gemini-3-1-pro/>, 2026.
- Danijar Hafner. Benchmarking the spectrum of agent capabilities. In *The Tenth International Conference on Learning Representations*, 2022.

- Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 16000–16009, 2022.
- Joy Hsu, Emily Jin, Jiajun Wu, and Niloy J Mitra. From programs to poses: Factored real-world scene generation via learned program libraries. *arXiv preprint arXiv:2510.10292*, 2025.
- Yushi Hu, Weijia Shi, Xingyu Fu, Dan Roth, Mari Ostendorf, Luke Zettlemoyer, Noah A. Smith, and Ranjay Krishna. Visual sketchpad: Sketching as a visual chain of thought for multimodal language models. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- Robert A Jacobs, Michael I Jordan, Steven J Nowlan, and Geoffrey E Hinton. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87, 1991.
- R Kenny Jones, Paul Guerrero, Niloy J Mitra, and Daniel Ritchie. Shapecoder: Discovering abstractions for visual programs from unstructured primitives. *ACM Transactions on Graphics (TOG)*, 42(4):1–17, 2023.
- R Kenny Jones, Paul Guerrero, Niloy J Mitra, and Daniel Ritchie. Shapelib: Designing a library of programmatic 3d shape abstractions with large language models. *arXiv preprint arXiv:2502.08884*, 2025.
- Herbert Kociemba. Kociemba’s solver. <https://kociemba.org/>, 2020.
- Chengzu Li, Wenshan Wu, Huanyu Zhang, Yan Xia, Shaoguang Mao, Li Dong, Ivan Vulić, and Furu Wei. Imagine while reasoning in space: Multimodal visualization-of-thought. *arXiv preprint arXiv:2501.07542*, 2025.
- Geng Li, Jinglin Xu, Yunzhen Zhao, and Yuxin Peng. Dyfo: A training-free dynamic focus visual search for enhancing lmms in fine-grained visual understanding. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 9098–9108, 2025.
- Jingming Liu, Yumeng Li, Boyuan Xiao, Yichang Jian, Ziang Qin, Tianjia Shao, Yao-Xiang Ding, and Kun Zhou. Enhancing visual reasoning with autonomous imagination in multimodal large language models. *arXiv preprint arXiv:2411.18142*, 2024.
- Tiange Luo, Lajanugen Logeswaran, Justin Johnson, and Honglak Lee. Visual test-time scaling for gui agent grounding. *arXiv preprint arXiv:2505.00684*, 2025.
- Aoran Mei, Guo-Niu Zhu, Huaxiang Zhang, and Zhongxue Gan. Replanvlm: Replanning robotic tasks with visual language models, 2024.
- OpenAI. Thinking with images. <https://openai.com/index/thinking-with-images/>, 2025.
- OpenAI. Introducing gpt-5.4. <https://openai.com/index/introducing-gpt-5-4/>, 2026.
- Wasu Top Piriyakulkij, Yichao Liang, Hao Tang, Adrian Weller, Marta Kryven, and Kevin Ellis. Poe-world: Compositional world modeling with products of programmatic experts. *arXiv preprint arXiv:2505.10819*, 2025.
- Hikaru Shindo, Viktor Pfanschilling, Devendra Singh Dhami, and Kristian Kersting. α ilp: thinking visual scenes as differentiable logic programs. *Machine Learning*, 112(5):1465–1497, 2023.
- Hikaru Shindo, Viktor Pfanschilling, Devendra Singh Dhami, and Kristian Kersting. Learning differentiable logic programs for abstract visual reasoning. *Machine Learning*, 113(11):8533–8584, 2024.
- Zhaochen Su, Linjie Li, Mingyang Song, Yunzhuo Hao, Zhengyuan Yang, Jun Zhang, Guanjie Chen, Jiawei Gu, Juntao Li, Xiaoye Qu, et al. Openthinking: Learning to think with images via visual tool reinforcement learning. *arXiv preprint arXiv:2505.08617*, 2025.
- Zhaochen Su, Peng Xia, Hangyu Guo, Zhenhua Liu, Yan Ma, Xiaoye Qu, Jiaqi Liu, Yanshu Li, Kaide Zeng, Zhengyuan Yang, et al. Thinking with images for multimodal reasoning: Foundations, methods, and future frontiers. *arXiv preprint arXiv:2506.23918*, 2025.

- Dídac Surís, Sachit Menon, and Carl Vondrick. Vipergpt: Visual inference via python execution for reasoning. In *ICCV*, pages 11854–11864, 2023.
- Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.
- Yikun Wang, Siyin Wang, Qinyuan Cheng, Zhaoye Fei, Liang Ding, Qipeng Guo, Dacheng Tao, and Xipeng Qiu. Visuothink: Empowering lvlm reasoning with multimodal tree search. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 21707–21719, 2025.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Penghao Wu and Saining Xie. V?: Guided visual search as a core mechanism in multimodal llms. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13084–13094, 2024.
- Hang Wu, Hongkai Chen, Yujun Cai, Chang Liu, Qingwen Ye, Ming-Hsuan Yang, and Yiwei Wang. Dimo-gui: Advancing test-time scaling in gui grounding via modality-aware visual reasoning. *arXiv preprint arXiv:2507.00008*, 2025.
- Yi Xu, Chengzu Li, Han Zhou, Xingchen Wan, Caiqi Zhang, Anna Korhonen, and Ivan Vulić. Visual planning: Let’s think only with images. In *The Fourteenth International Conference on Learning Representations*, 2026.
- Qingqing Zhao, Yao Lu, Moo Jin Kim, Zipeng Fu, Zhuoyang Zhang, Yecheng Wu, Zhaoshuo Li, Qianli Ma, Song Han, Chelsea Finn, Ankur Handa, Ming-Yu Liu, Donglai Xiang, Gordon Wetzstein, and Tsung-Yi Lin. Cot-vla: Visual chain-of-thought reasoning for vision-language-action models, 2025.
- Ziwei Zheng, Michael Yang, Jack Hong, Chenxiao Zhao, Guohai Xu, Le Yang, Chao Shen, and XingYu. Deepeyes: Incentivizing ”thinking with images” via reinforcement learning. In *The Fourteenth International Conference on Learning Representations*, 2026.

A More Experimental Results

A.1 OOD Generalization

In short: patterns naturally share across different sizes of the same kind of experiments, but weights don’t always.

We want to test if, after getting trained on smaller maps, the patterns and weights could zero-shot generalize to larger maps. This means that not only are the patterns fixed, but also the weights are fixed. And they will not be trained on larger maps.

We tested CRAFT’s learned pattern & weights from 64x64 map on a larger 128x128 map. The result shows that patterns & their weights have strong OOD generalization capability. The reveal count drops from 2349.63 to 1570.45, indicating its effectiveness.

A.2 More Qualitative Results

Qualitative illustration of the online inductive learning process under FROZENLAKE and CUBEBENCH are illustrated in Fig. 4 and Fig. 5.

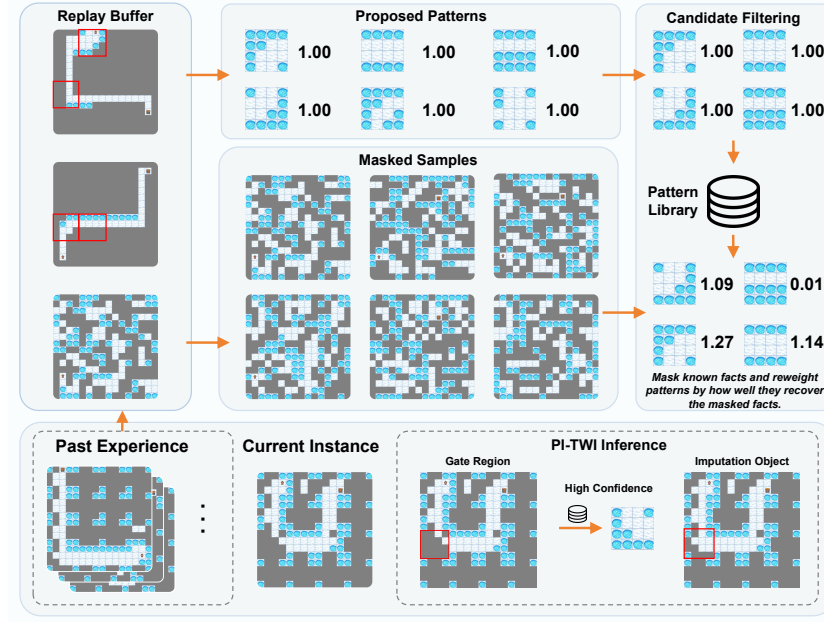


Figure 4: Qualitative illustration of the pattern inference and induction processes in FROZENLAKE.

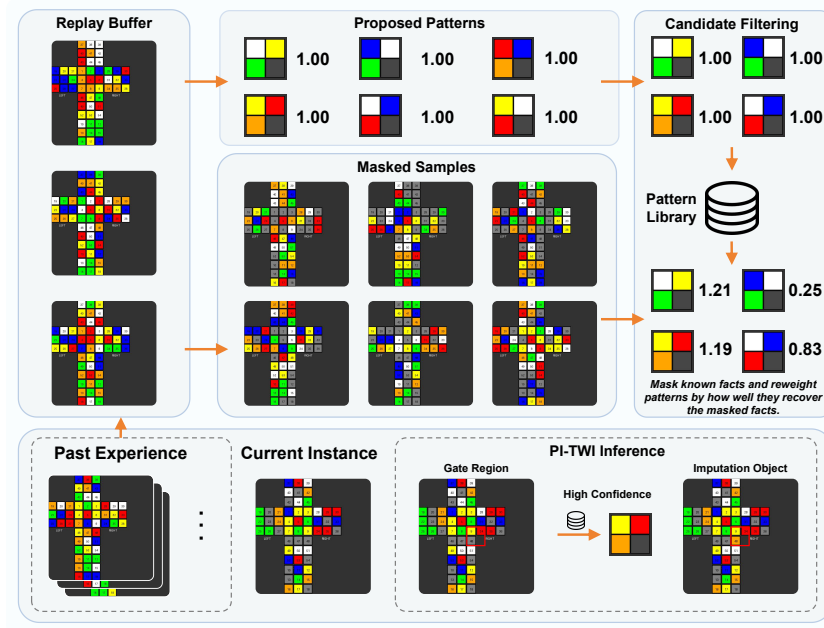


Figure 5: Qualitative illustration of the pattern inference and induction processes in CUBEBENCH.

B Implementation Details

Our experiments are conducted on a personal computer with Mac M3 Max CPU (16 cores) and 128GB RAM. For VLM calls, we utilize OpenRouter³ and OpenAI API Platform⁴.

³<https://openrouter.ai/>

⁴<https://platform.openai.com/>

We also modify CRAFT and FROZENLAKE environments, which are both built upon Gymnasium [Towers *et al.*, 2024] framework, to fit our purposes.

There are 100 episodes for each run on each benchmark. For each benchmark, we conduct a total of 9 runs. Specifically, we employ 3 seeds to initialize distinct environment instances (map or state sequences). For each seed, 3 trials are performed to ensure statistical robustness. The results in Tab. 1 and Tab. 2 are mean results over runs.

Relation between token consumption and reveal count. We use total token consumption as the primary efficiency metric throughout the paper. Reveal count is reported only as a diagnostic quantity to explain where the reduction in *perception* token consumption comes from. In our implementation, each reveal operation within the same task uses a fixed prompt template, image resolution, and output schema. We therefore account for perception-token usage with a fixed per-reveal token budget:

$$T_{\text{perc}} = c_{\text{perc}} \cdot N_{\text{perc}},$$

where T_{perc} is the perception token count, N_{perc} is the reveal count and c_{perc} is the task-specific token budget per reveal operation. The total token cost additionally includes proposal tokens, so all main efficiency comparisons are still reported using total token consumption.

For FROZENLAKE and CUBE BENCH, the per-reveal perception costs are fixed: 98 input and 5 output tokens for FROZENLAKE, and 88 input and 5 output tokens for CUBE BENCH. For CRAFT, the input cost is fixed at 184 tokens, while the output cost can vary slightly from 3 to 5 tokens depending on the predicted cell type. We therefore use 5 output tokens as a conservative per-reveal budget for CRAFT. This convention provides an upper-bound estimate of perception-token usage; since the variation is at most two output tokens per reveal and is negligible compared to the input-token cost, it does not affect the relative efficiency comparison.

An example: In PI-TWI’s “w/o inference” ablation for CUBE BENCH, the 4.75k input perception tokens come from 54 reveal operations (since a Rubik’s cube has 54 facelets), namely $54 \times 88 = 4752$.

B.1 Policy-generation procedure

FROZENLAKE This environment follows LAZYSP [Dellin and Srinivasa, 2016] shortest path algorithm. Simply put, the LAZYSP algorithm is like this: at first, we only know the locations of the start and the goal, but not any passable “ground” or impassable “hole”. We plan optimistically:

1. We assume every unrevealed grid is passable. On top of this, we plan the shortest path.
2. If there aren’t any unrevealed grids on the path, the algorithm stops and returns the path
3. If there are, we check the unrevealed grids one by one from start to goal.
4. If an impassable grid is ever encountered during this process, we immediately go back to step 1 to get another plan.
5. If all checked grids are passable, the algorithm stops and returns the path.

The *policy-generation procedure* works at step (4) above. Each time, it outputs the first unrevealed grid from start to goal.

CRAFT This environment, just like the original Minecraft, contains many achievements, The success condition is to finish all 14 achievements in Fig. 6.

Inspired by task and motion planning, we treat each “achievement” as a *task*. Since there are dependencies between these tasks (Fig. 6), we can get its topological order (see C.2) and finish tasks one by one with this order.

And for each specific task, we have to gather enough materials to craft a certain tool. In order to gather materials, we have to first find the required materials for this specific task, and plan a feasible route to it. Here, we design an algorithm that do both at the same time. This is where the *policy-generation procedure* G works: it finds all passable grids among the revealed grids ⁵, and outputs all their

⁵“grass”, “sand” and “path” are natural passable grids; “tree” is also passable, since you can mine it with your bare hands and pass it; if you have wooden pickaxe, “stone” and “coal” will also become passable, since they are now minable; etc.

adjacent unrevealed grid. Intuitively, this G gradually expand the boundary of revealed grids, while making sure all passable revealed grids are connected. In this way, we can make sure if we find a certain type of material, we can have at least one feasible route towards it.

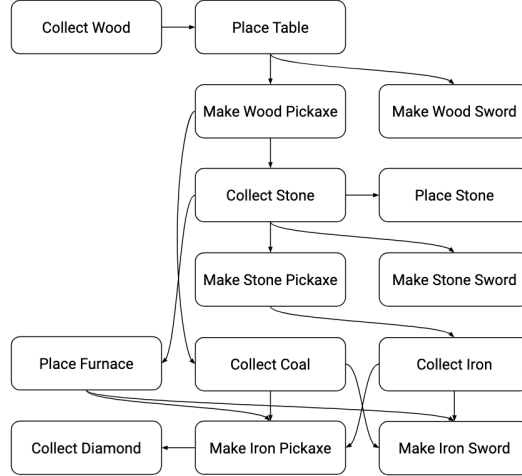


Figure 6: Achievements for CRAFT

CUBEBENCH We adopt the environment and solver interface from CubeBench [Gao *et al.*, 2025]. In this setting, the input is an image observation of a scrambled Rubik’s Cube, and the VLM is asked to convert the visual observation into a complete 54-character symbolic state. Each character denotes one sticker color using the CubeBench color alphabet R, G, B, Y, O, and W, ordered according to the CubeBench simulator convention.

After the VLM outputs the 54-character state code, the code is passed to the CubeBench solver interface. The solver is used as a black-box symbolic backend that returns a sequence of cube moves for the predicted state. Our focus is therefore on obtaining an accurate symbolic state from the image before invoking the solver, not on modifying or analyzing the solver’s internal search procedure.

B.2 Pattern Proposal

Since FROZENLAKE is 16x16 and CRAFT is 64x64, if you put the entire symbolic world model into the prompt, it will contain too many tokens, and VLMs will fail to generate good proposals in such long context.

So, we need to preprocess the symbolic world models from the replay buffer.

FROZENLAKE

1. We partition each 16x16 map in the replay buffer into 16 non-overlapping 4x4 minimaps.
2. Minimaps containing fewer than 8 revealed grids are discarded to ensure sufficient information density.
3. We randomly sample 5 minimaps from the remaining candidates.
4. These 5 minimaps are subsequently integrated into our prompt template.

This proposal will be triggered every 5 maps. So for in total 100 episodes, there will be 19 proposals.

CRAFT

1. For each 64x64 map in the replay buffer, we identify the minimal bounding box enclosing all revealed grids and subsequently crop the map to these boundaries.

Basically, a grid being passable means you can either directly walk through it, or do so after using your existing tools like pickaxe.

2. Each cropped map is subdivided into multiple 15x15 minimaps. To accommodate cross-shaped patterns, these minimaps maintain at least a 3x3 overlap. We optimize the minimap locations such that the maximum overlap is minimized.
3. We randomly sample 5 minimaps from the resulting set.
4. These 5 minimaps are integrated into our prompt template.

This proposal will be triggered every 5 maps. So for in total 100 episodes, there will be 19 proposals.

CUBEBENCH

1. We select unreflected states from the replay buffer, prioritizing failed reconstructions and using successful reconstructions only when additional examples are needed.
2. For each selected state, we extract the eight corner cubies and represent each corner by its three-color token.
3. We add the corner cubie-to-sticker-index mapping and a deduplicated summary of observed corner tokens with their multiplicities.
4. These examples are integrated into our prompt template, and the model proposes single-corner patterns in strict JSON format.

This proposal will be triggered every 10 episodes. So for in total 100 episodes, there will be 9 proposals, since no proposal is needed after the final episode.

The prompts are listed below. The minimaps will be filled into each “{PARTIAL_MAP_BLOCK}” in the prompt. The “{K}” will be 10 for CRAFT, 10 for FROZENLAKE and 3 for CUBEBENCH.

Prompt for FrozenLake Skill Proposal

Task Description

You will be provided with symbolic partial maps or aligned tiles from a FrozenLake-style environment.

Each map uses three labels:

- SAFE: observed traversable cell
- HOLE: observed unsafe cell
- UNKNOWN: unobserved cell

Your goal is to infer up to {K} reusable 4x4 local navigation skill patterns from the provided examples.

Grid Alignment Rules

Full maps are divided into non-overlapping aligned 4x4 blocks; smaller provided examples may already be aligned tiles.

Only these aligned 4x4 blocks may be considered.

Do not use sliding windows.

Do not shift, crop, rotate, resize, or merge blocks.

Pattern Guidelines

- * Look for paths: Ideal local patterns resemble safe structures like corridors or turns surrounded by holes.
- * Prioritize reusability: Look for common, generalizable shapes rather than copying one-off observations.
- * No unknowns: Every cell in your final patterns must be labeled exactly "SAFE" or "HOLE".

Note even if there are "UNKNOWN"s in the examples, you are still pretty much able to infer and guess.

Strict Output Format

Return your answer as valid JSON ONLY. Do not provide conversational text or Markdown code blocks outside the JSON.


```

Structure:
{
  "patterns": [
    [
      ["R1C1", "R1C2", "R1C3", "R1C4"],
      ["R2C1", "R2C2", "R2C3", "R2C4"],
      ["R3C1", "R3C2", "R3C3", "R3C4"],
      ["R4C1", "R4C2", "R4C3", "R4C4"]
    ]
  ]
}

### Example {i}
{PARTIAL_MAP_BLOCK}

```

Prompt for CRAFTER Pattern Proposal

You are given partially revealed Crafter material maps.
 Crafter is essentially 2D Minecraft; so also use that world knowledge to infer.

Each pattern should use top/bottom/left/right to predict the center.
 Propose at most {K} reusable cross-shaped patterns.
 Return strict JSON only with shape {"patterns": [{"center": "...", "top": "...", "bottom": "...", "left": "...", "right": "...}]}.

Use only lowercase world-generation material names. Do not use unknown.

Example {i}
 episode={episode_id}
 bbox: x=[{bbox_xmin},{bbox_xmax}], y=[{bbox_ymin},{bbox_ymax}]
 {PARTIAL_MAP_BLOCK}

Prompt for CUBEENCH Pattern Proposal

You are extracting reusable corner-cubie skills from pre-extracted corner observations.

Each replay example already contains all 8 corner cubies (CORNERS_8).
 Character alphabet for each corner token is exactly {R,G,B,Y,O,W}.

Only use corner cubies: {corner_cubie_names}.

Corner cubie-to-sticker mapping:
 {corner_cubie_to_sticker_mapping}

Task:

- Read CORNERS_8 examples.
- Extract fully-specified corner tokens as <CUBIE>:<TOKEN>.
- Consolidate duplicates by identical <CUBIE>:<TOKEN> pairs (same pair can appear in multiple examples).
- Each proposed pattern must contain exactly one corner cubie.
- Corner token length must be exactly 3.

Replay examples:
 {replay_examples}

Deduplicated corner token summary (with multiplicity):
 {deduplicated_corner_token_summary}

Propose at most {patterns_per_trigger} new reusable skills.
 Return strict JSON only with shape:

```
{
  "patterns": [{"cubies": {"URF": "ROW"}}, ...]
}
Do not add commentary. Do not use markdown.
'''
```

Here ‘{corner_cubie_names}’ is the list of corner cubie identifiers, ‘{corner_cubie_to_sticker_mapping}’ lists each corner cubie’s simulator-order sticker indices with face names, ‘{replay_examples}’ contains the selected ‘CORNERS_8’ states, and ‘{deduplicated_corner_token_summary}’ contains the unique ‘<CUBIE>:<TOKEN>’ pairs with multiplicities.

B.3 Imputation

In addition to the pattern-based imputation rule introduced in Sec. 3.2, there are two hyperparameters worth noting: the confidence threshold τ and the smoothing constant ε . The threshold τ controls how conservative the agent is when trying to add pattern-imputed facts into K_t^I . A larger τ reduces the risk of imputing incorrect symbolic facts into the world model. The smoothing constant ε , which is originally used in Eq. 8 to avoid assigning zero probability to non-predicted values, can also affect the conservativeness. We use a fixed $\varepsilon = 0.001$ across all domains.

Domain	τ	ε
FROZENLAKE	0.99	0.001
CRAFTER	1.00	0.001
CUBEBENCH	0.99	0.001

B.4 Reranking

Reranking uses the same pattern-predicted distribution $p_w(y_u | M_t)$ as imputation, but it does not add any unrevealed facts to the world model. Instead, it only changes the order in which candidate variables from $G(M_t)$ are revealed. Therefore, reranking can exploit stochastic regularities without just imputing them as hard symbolic facts.

In our experiments, reranking is used in CRAFTER, where $G(M_t)$ often returns many adjacent unrevealed visual variables on the current frontier. For a resource-acquisition subtask with the set of target resource types Z , we score each candidate variable $u \in G(M_t)$ by

$$s(u) = p_w(y_u \in Z | M_t),$$

and reveal the candidate with the highest score. This allows the agent to spend perception calls preferentially on variables that are more likely to contain task-relevant resources, such as trees, stone, coal, iron, or diamond, while still requiring direct perception before any such variable is added to the world model.

B.5 Weight Optimization

Since the max number of patterns in the library is relatively small, we choose L-BFGS as our gradient-based optimization method for faster convergence.

For FROZENLAKE, the learning rate is 1.0 and the maximum iteration is 50. For CRAFTER, the learning rate is 1.0 and the maximum iteration is 20. For CUBEBENCH, the learning rate is 1.0 and the maximum iteration is 150.

C Benchmark Details

We introduce three benchmarks in our experiments. (1) **FrozenLake** [Towers *et al.*, 2024]. The agent observes a rendered grid world and must find a shortest safe path from the start to the goal without entering holes. Each visual variable is a grid cell, and its value is the cell type. (2) **Crafter**. [Hafner, 2022]. CRAFTER is originally a stochastic survival environment. We repurpose it as a deterministic visual resource acquisition and crafting task: the PI-TWI agent is given the task specification, and

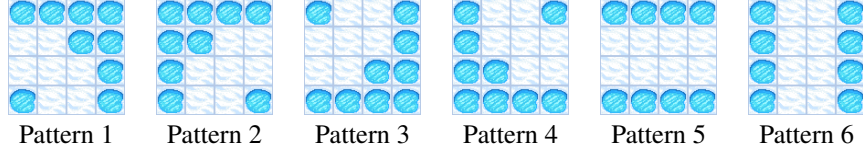


Figure 7: The six manually designed 4x4 macro patterns used to generate the modified FROZENLAKE maps.

must ground enough of the map to plan a feasible sequence of navigation, collection, and crafting actions. Each visual variable is a grid cell, and its value is the corresponding terrain, object, or resource type. (3) **CubeBench**. [Gao *et al.*, 2025]. We evaluate cube-state grounding as a structured visual planning problem. Each instance renders a cube state; the agent must reconstruct enough symbolic facelet information for a downstream cube planner or solver. Each visual variable is a facelet and its value is one of the six colors.

There are 100 episodes for each run on each benchmark.

We repurpose FROZENLAKE and CRAFTED to make them suitable for our method.

C.1 FROZENLAKE

Action space: move up, move down, move left, move right.

Originally, for each grid in this environment, it has a fixed probability to become a “hole”. So the map is randomly generated and does not have any patterns.

In our modified version, we designed six 4x4 macro patterns (Fig. 7). We also make sure the shortest path from start to goal is above a minimum. Then, we generate a map with this process:

1. First divide the 16x16 map into 16 4x4 minimaps
2. For each minimap, we randomly put one of six 4x4 macro patterns there
3. We randomly sample the start and goal
4. If the length of the shortest path in between is less than the minimum, we discard this map and go back to step (1); otherwise, we return this map.

Our experiments are done with the minimal length of 25.

This generation method is basically rejection sampling, ensuring more uniform distribution.

However, for our OOD test, where we need to generate 32x32 maps with minimal length of 50, the efficiency is too low, so we adopt another method:

1. First divide the 16x16 map into 16 4x4 minimaps
2. For each minimap, we randomly put one of six 4x4 macro patterns there
3. We calculate the length of shortest path between all pairs of grids using `all_pairs_shortest_path_length` from `networkx`
4. We filter all pairs whose lengths are smaller than minimum
5. If there’s no pairs left, we discard this map and go back to step (1); otherwise, we uniformly sample one pair as start and goal, and return the map.

Essentially, this is still rejection sampling, but with much better efficiency.

C.2 CRAFTED

The original CRAFTED has 17 actions available. For our purpose, we removed action Noop and Sleep. The remaining actions are: move left, move right, move up, move down, do, place stone, place table, place furnace, place plant, make wood pickaxe, make stone pickaxe, make iron pickaxe, make wood sword, make stone sword and make iron sword.

Among the many game mechanics, we removed: (1) Spawned mobs (zombie, skeleton) and creatures (cow) (2) Moving objects (arrow), and disable: (1) Hunger (2) Thirst (3) Energy (4) Daylight cycle. We remove these two stochastic elements because stochasticity will make TWI for open-loop planning almost impossible. For similar reasons, we also disable exogenous mechanics like hunger and thirst.

There are originally 22 achievements. We remove 8 of them, which are:

1. **Collect Drink:** Removed because the thirst mechanic is disabled.
2. **Defeat Zombie & Defeat Skeleton:** Removed because mob spawning is disabled.
3. **Eat Cow:** Removed because both hunger mechanic and animal spawning are disabled.
4. **Wake up:** Removed because the daylight cycle mechanic is disabled.
5. **Collect Sapling, Place Plant, & Eat Plant:** While these could technically be integrated into the planner, they do not serve our research goals. Since the player always spawns on grass, these tasks can be completed through repetitive actions in a single location (e.g., harvesting grass until a sapling drops). Because they require no environmental exploration or navigation, they offer no meaningful challenge to the agent’s spatial perception.

Since we no longer have to collect saplings, we also set the probability of getting a sapling from grass to zero.

The remaining 14 achievements will be done one by one in this order, which is one of the possible topological order of the DAG of the achievements:

collect wood → place table → make wood pickaxe → make wood sword → collect stone → place stone → make stone pickaxe → make stone sword → place furnace → collect coal → collect iron → make iron pickaxe → make iron sword → collect diamond

When generating a map, we will make sure each map gets enough resources to complete all 14 achievements. In total, we need 9 trees, 8 stones, 3 coal ores, 3 iron ores and 1 diamond ore. If one map is generated without enough resources, we will discard and regenerate the map.

C.3 CUBE BENCH

The original CubeBench task takes an image observation of a scrambled Rubik’s Cube, asks the model to recover the complete 54-character symbolic cube state, and then passes that state to the Kociemba’s Solver that returns a sequence of cube moves. For our purpose, we keep the CubeBench state representation, color alphabet, simulator-order indexing, and solver interface, but remove the final cube-solving action sequence from the main evaluation target. Since an incorrect reconstructed state makes the downstream solve plan meaningless, we focus on the partial-observation state reconstruction problem before solver execution.

We replace one-shot visual state prediction with an active reveal protocol. A rollout begins with all 54 stickers unknown; each round actively reads one ground-truth sticker and then runs pattern-assisted imputation over the remaining unknown stickers. Inferred stickers do not count as active reveals. The experiment data is generated locally from CubeBench: the default configuration creates 100 states with 20 random scramble moves and seed 42, and saves the state list, scramble metadata, global cubie map, and sticker adjacency table. We fix the evaluated states and scramble seed so that performance differences can be attributed to active information gathering, pattern coverage, LLM-proposed skills, and learned weighting rather than dataset-generation stochasticity.

We evaluate two metrics: exact reconstruction accuracy, where all 54 predicted stickers must match the ground truth, and active reveal cost, which counts only stickers explicitly read from the ground truth. By default, the runner evaluates 100 states from the generated dataset. Pattern libraries are grouped by source, including ground-truth patterns, irrelevant patterns, placeholder patterns, and LLM-proposed patterns. The default placeholder library uses one-unknown corner templates with 24 unique candidates for each corner cubie position, and the current LLM proposal implementation accepts exactly one fully specified corner cubie per proposed pattern. The configuration also supports ablations that disable inference, reweighting, placeholders, ground-truth patterns, or LLM proposal, separating the effects of active imputation, placeholder coverage, proposal, and learned weighting.

C.4 Macro Patterns

A macro pattern in FROZENLAKE can be seen in Fig. 7. One macro patterns corresponds to 16 patterns, where the prediction of each pattern is one of the grid among the $4 \times 4 = 16$ grids, and the gate of each pattern is composed of other 15 grids.

Similarly a macro pattern of CubeBench corresponds to 3 patterns, where the prediction of each pattern is one of the facelet among the 3 facelets, and the gate of each pattern is composed of other 2 facelets.

The macro pattern of CRAFTER corresponds to only one pattern, where the prediction is the center grid and the gate is composed of the four surrounding it.

C.5 Ground-Truth Planner

In each domain, the ground-truth planner is instantiated by LazySP [Dellin and Srinivasa, 2016] for FROZENLAKE, by a task-and-motion-planning-style search procedure [Garrett *et al.*, 2021] for CRAFTER, and by Kociemba’s Solver [Kociemba, 2020] for CUBEENCH. Note that in FROZENLAKE, the planner and grounding controller G are the same due to the native grounding guidance ability of the planners.

D Baseline Details

D.1 Direct VLM Output

For the “Direct VLM Output” experiment, prompts are provided directly to the model to elicit feedback. Note that while the first image in each prompt is an episode-specific map that varies, all other images in the prompts are static and consistent across experiments.

D.2 Native TWI

For the “Native TWI” experiment, prompts are given to the native TWI agent for each model. For GPT-5.4, the agent is called by “code interpreter” via OpenAI API; for Gemini 3.1 Pro, it is “Code execution”; for Qwen3 VL 235B A22B, it is the official “think_with_images” Jupyter notebook of QwenLM/Qwen3-VL repository on Github. The prompts are similar to that of “Direct VLM Output”. The main difference is that here we ban it from using any tools other than those for the purpose of cropping images, since other methods like using Python programs to analyze the pixel of a grid and output its type aren’t in our consideration. If the agent does use Python in a specific task, it will be considered as plagiarism and fail this task.

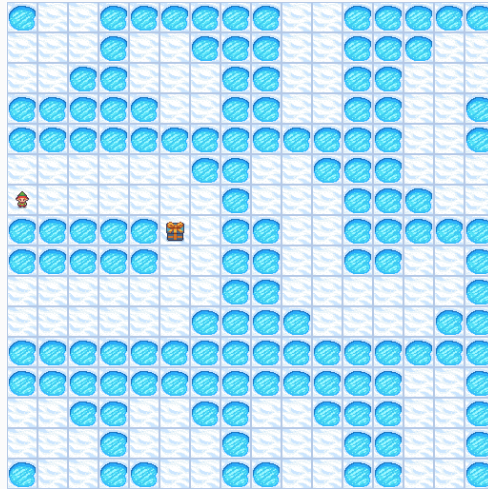
E Potential Societal Impact

Our work is purely about improving visual planning abilities of VLMs. We do not find any significant societal impact requiring to be explicitly discussed.

F Prompts for Baselines

We provide the prompts for the baseline methods of direct VLM output and native TWI.

Prompt for FROZENLAKE Direct VLM Output



Player



**Hole
(Not safe)**



**Land
(Safe)**



Goal

Output the 16x16 map of this image, with the help of the reference. Suppose left-top is (0,0), and bottom-right is (15, 15). (row, col)

You only need to output all coords of holes.

Format:

```
““json
{"Holes": [(..., ...), ...]}
““
```

Prompt for CRAFT Direct VLM Output

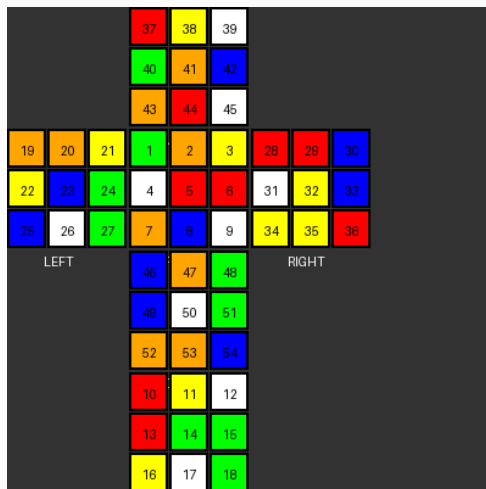


Output the 64x64 map of this image, with the help of the reference. Suppose left-top is (0,0), and bottom-right is (63, 63). (row, col)

Format:

```
““json
{"sand": [..., ...], "grass": [...], ...}
““
```

Prompt for CUBE BENCH Direct VLM Output



You are a Rubik's Cube state reader. The image shows each sticker with a global numeric ID.

Read sticker colors by ID from 1 to 54 and build 'state' as this exact ID order.

[Color codes]

Only use: R, G, B, Y, O, W

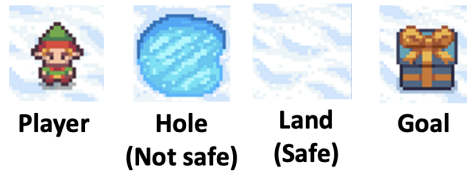
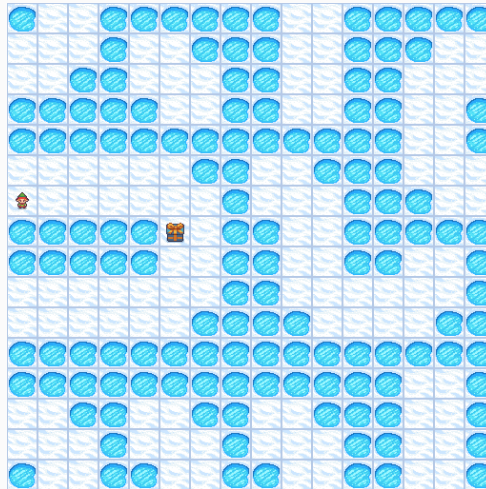
[Strict output requirements]

- * Output only one JSON object
- * No explanation, no markdown, no extra text
- * state must be exactly 54 characters
- * state must be exactly colors of ID 1..54 in order
- * state characters must be only R/G/B/Y/O/W

[Allowed output format]

```
{"state": "XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"}
```

Prompt for FROZENLAKE Native TWI



Output the 16x16 map of this image, with the help of the reference. Suppose left-top is (0,0), and bottom-right is (15, 15). (row, col)

You only need to output all coords of holes.

Format:

```
““json
{"Holes": [(..., ...), ...]}
““
```


Warning: You cannot use any tools other than vanilla "image cropping" to analyze the image. You are especially banned from using any coding besides the simplest cropping without any further visual cues.

This means, besides basic system commands like 'cd', 'mkdir', 'ls', etc., the only Bash command you can use is 'magick' to crop the image. Also, you are completely banned from running 'python' or any kind of popular programming language script.

Prompt for CRAFTER Native TWI



Output the 64x64 map of this image, with the help of the reference. Suppose left-top is (0,0), and bottom-right is (63, 63). (row, col)

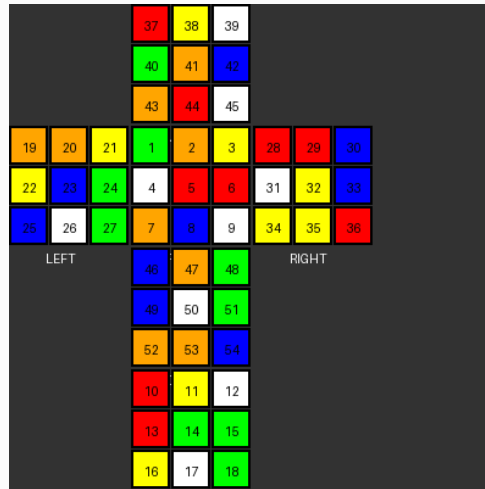
Format:

```
““json
{"sand": [(..., ...), ...], "grass": [...], ...}
““
```

Warning: You cannot use any tools other than vanilla "image cropping" to analyze the image. You are especially banned from using any coding besides the simplest cropping without any further visual cues.

This means, besides basic system commands like 'cd', 'mkdir', 'ls', etc., the only Bash command you can use is 'magick' to crop the image. Also, you are completely banned from running 'python' or any kind of popular programming language script.

Prompt for CUBE BENCH Native TWI



You are a Rubik's Cube state reader. The image shows each sticker with a global numeric ID.

Read sticker colors by ID from 1 to 54 and build 'state' as this exact ID order.

[Color codes]

Only use: R, G, B, Y, O, W

[Strict output requirements]

- * Output only one JSON object
- * No explanation, no markdown, no extra text
- * state must be exactly 54 characters
- * state must be exactly colors of ID 1..54 in order
- * state characters must be only R/G/B/Y/O/W

[Allowed output format]

```
{"state":"XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX"}
```

Warning: You cannot use any tools other than vanilla "image cropping" to analyze the image. You are especially banned from using any coding besides the simplest cropping without any further visual cues.

This means, besides basic system commands like 'cd', 'mkdir', 'ls', etc., the only Bash command you can use is 'magick' to crop the image. Also, you are completely banned from running 'python' or any kind of popular programming language script.